01 · 프로젝트 개요 · 2026.03 ~ 운영 중

출입 기록 기반 학습 리포트 서비스

매장 관리 시스템의 출입 로그를 매일 수집해 회원별 학습 세션으로 변환하고 일·주·월 리포트로 제공

기획 배경

이용자 본인 학습 기록 확인 수요
운영자 학습 관리 인력 부재

→ 매장에 이미 쌓이는 출입 기록 활용

데이터 특성

출입 로그에는 입실·외출·퇴실 시각만 존재하고, 학습 시간 정보는 없음

→ 회원별 이벤트를 시간순으로 묶어 학습 세션 생성 (세션화)



파이프라인

매장 관리 시스템 → 일 1회 크롤링 → 원본 저장 → **세션화** → 일·주·월 리포트

SQL(PostgreSQL) · GitHub Actions · TypeScript · Playwright · React

02 · 데이터 품질 검사 · 정제 규칙 재설계

23:59 보정 세션을 검사하고, 하루의 경계를 04시로 옮겼다

기존 보정 방식

자정 넘김 23:59:59 퇴실 + 익일 00:00 가상 입실로 분할 저장

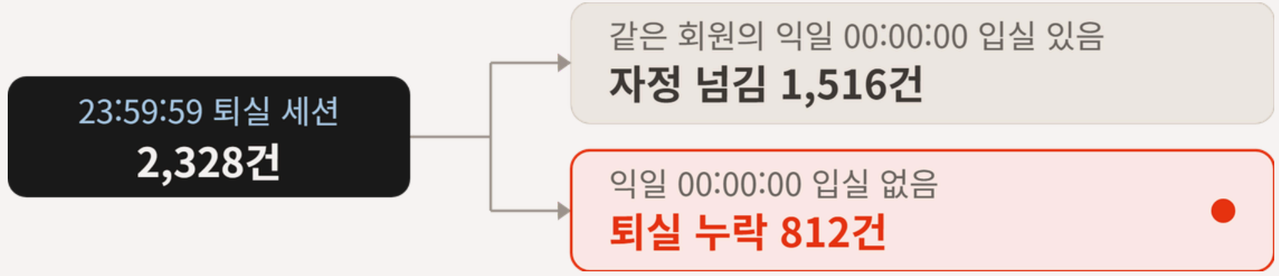
퇴실 미기록 23:59:59 퇴실로 채움

→ **두 경우 모두 23:59:59 퇴실로 저장되어 구분 불가**

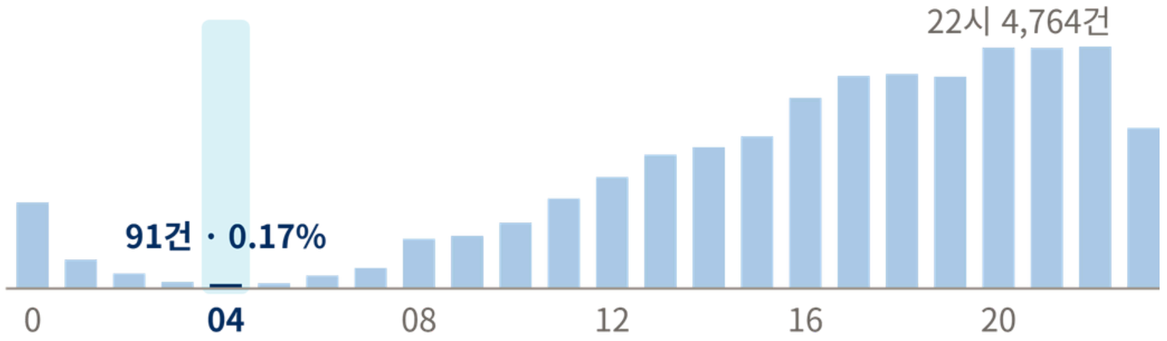
기록상 학습 시간 중간값

퇴실 누락 **304분** 정상 퇴실 **158분**

퇴실을 찍지 않은 세션은 실제로 나간 시각을 알 수 없어, 기록 시간이 실제보다 길게 잡혔을 수 있음



하루의 경계를 04시로 — 시간대별 출입 기록 52,981건 중 04시대가 91건(0.17%)으로 가장 적어, 이 시각을 하루의 경계로 삼음



SQL(PostgreSQL) · GitHub Actions · TypeScript · Playwright · React

v2 검증 중 · 운영 미반영

하루 경계 변경 전후 세션 비교

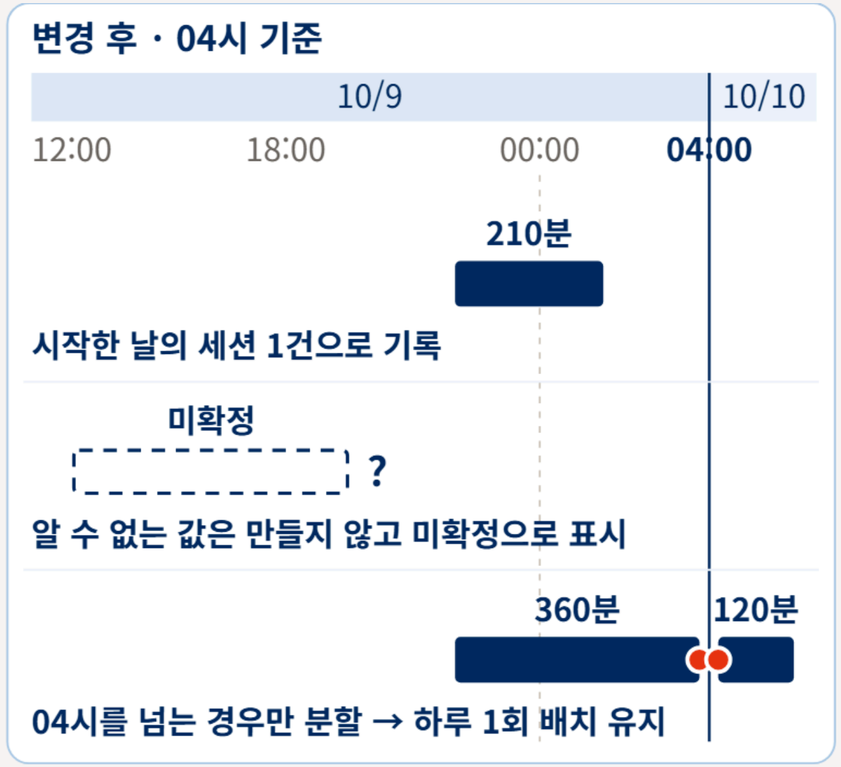
하루의 범위
하루를 자르는 기준을
출입이 가장 적은 04시로

사례 1 · 자정을 넘긴 이용
가장 흔한 경우
22:00 입실 → 01:30 퇴실

사례 2 · 퇴실 기록 없음
실제 이탈 시각 모름
13:00 입실 → 퇴실 미기록

사례 3 · 04시를 넘긴 밤샘
드문 경우
22:00 입실 → 익일 06:00 퇴실





분석 기간(2026.03~10) 중 04시 이후 퇴실은 **19건**(회원-일 기준 추정)으로, 04시 기준에서 분할되는 경우는 드묾

● 시스템이 만든 가상 입실·퇴실

SQL(PostgreSQL) · GitHub Actions · TypeScript · Playwright · React

04 · 수집 품질 게이트

품질 규칙 기반 수집 배치

도입 배경

4/30, 5/14 세션 0건이었으나 알림 없음

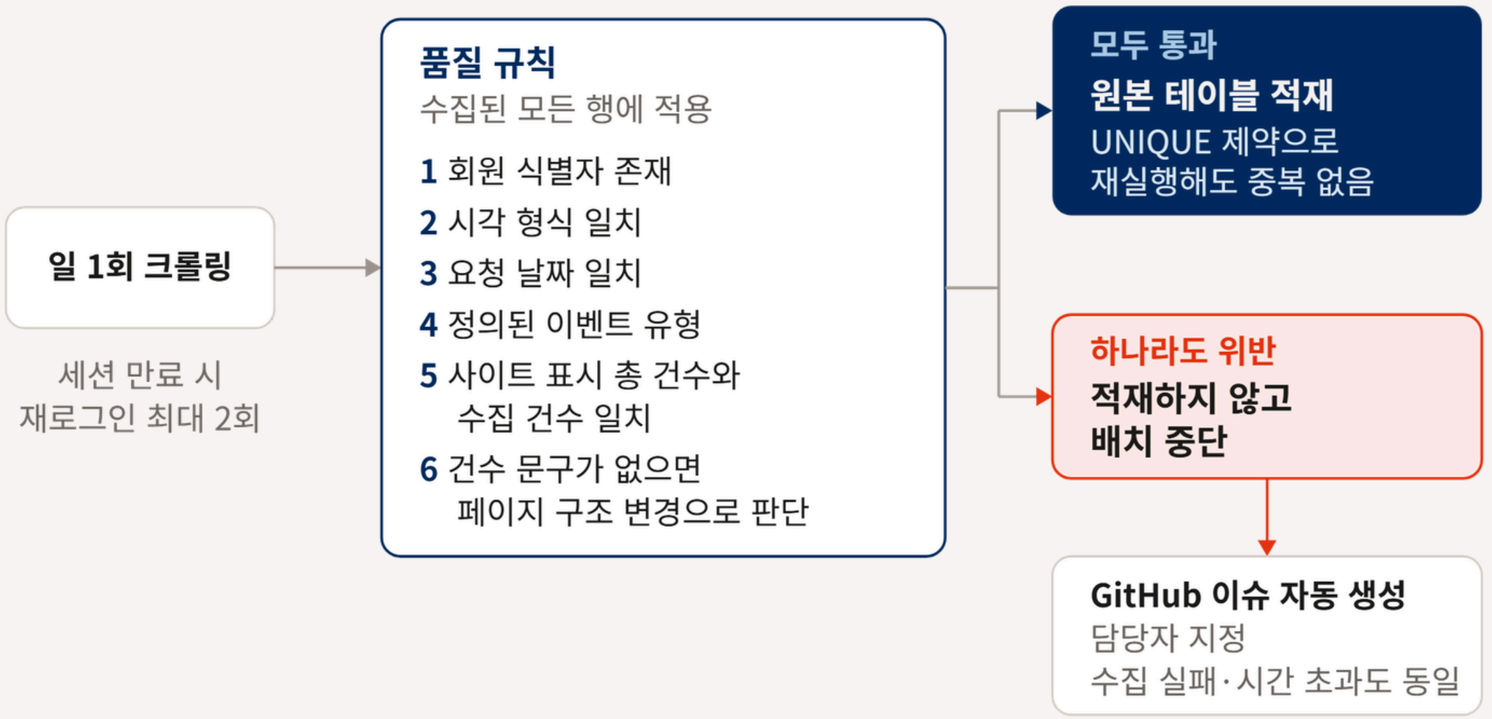
형식 오류 행은 건너뛰고, 저장 오류는 로그만 남김

→ **배치가 실패해도 성공으로 종료**

현재까지 확인된 것

알림 동작 테스트 완료 (10/3)

도입 후 실제 실패 감지 기록 없음 (운영 기간이 짧음)



SQL(PostgreSQL) · GitHub Actions · TypeScript · Playwright · React

Qode 01 · 프로젝트와 내 역할 · 2026.02 – 2026.09 [프론트 1명, 백엔드 1명, RAG 1명]

Qode

비개발자가 코드 근거를 직접 확인하는 AI 협업 도구

서비스

기획자 같은 비개발자가 자기 말로 질문하면 관련 코드를 찾아 근거 위치와 함께 답하고, 필요한 내용을 개발자에게 확인 요청으로 공유

질문 → **근거 포함 답변** → **개발자에게 공유**

비개발자의 말로 ▲ **RAG 검색 품질** ▲ **개발자 공유 기능**

내 역할

RAG 검색 품질

품질평가 기준과 평가 데이터를 설계하고, 검색 실패 원인을 단계별로 진단·개선

개발자 공유 기능

비개발자의 질의와 근거를 개발자에게 전달하는 데이터 구조와 공유 흐름 구현

비개발자가 코드 근거를 믿고 쓰려면, 먼저 검색이 정답 코드를 찾아야 한다

“필요한 코드 근거를 정확히 찾아낼 수 있는가”부터 검증했다

Qode 02 · 지식 품질평가 기준 설계

검색 품질을 판단할 정답 기준을 직접 정의했다

평가 대상	**httpx 레포** — 구조가 분명해 정답을 원문과 대조하기 쉬움
질문 유형	What · Where · How · Why 각 20문항, 총 **80문항**
재현성	정답 근거를 커밋이 고정된 원문 위치(파일·심볼·줄 범위)로 기록
근거 등급	**primary**(이유를 서술한 곳) / **supporting**(구현된 곳)
범위 정의	외부 패키지는 정답에서 제외하고 따로 기록
작성·검수	질문 80개 직접 작성. 정답·근거는 **16문항 직접**, 64문항은 AI 초안 후 원문 대조 검수
다음 단계	**비개발자 질의 세트** — 같은 정답을 비개발자 표현으로 바꾼 질의 구성 중

평가 데이터 예시 (JSON 축약)

```
question:
  리다이렉트를 거치며 이전 응답 기록은 어디에 쌓이지?
category: where
reference_answer:
  response.history 리스트에 순서대로 쌓임
golden_source:
  - httpx/_client.py
    Client._send_handling_redirects · L964-992
  - httpx/_models.py
    Response.history · L516-541
```

파일 / 심볼 / 줄 범위 recall
정답 근거를 어느 정밀도까지 찾았는가

all_found · MRR
필요한 근거를 모두 찾았는가 / 첫 정답의 순위

NormRecall@k
정답 근거가 k개보다 많으면 분모를 k로 맞춘 recall

평가 오류를 바로잡고 조각을 다듬었지만, 조각 단위 개선으로는 recall이 오르지 않았다

먼저 바로잡은 것 · 평가 오류

팀 합의대로 벡터 DB에서는 문서(.md)를 뺐는데, 정답 세트에서는 문서 근거를 빼지 않고 평가 **(내 실수)**

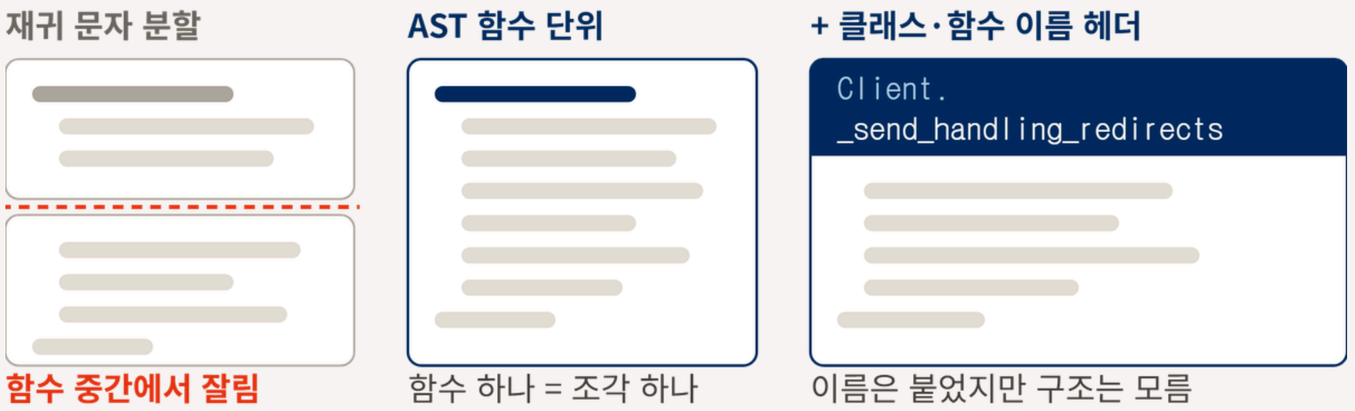
→ 검색으로 찾을 수 없는 정답까지 채점해 recall이 낮게 나옴

→ 정답에서도 문서 근거를 제외하고 다시 평가

조각을 잘 자르고 이름을 붙여도 질문과의 거리가 줄지 않았다

조각 하나만으로는 그 코드가 어떤 구조 안에서 의미를 갖는지 알 수 없다

단계	조각 단위 개선 시도	NormRecall@10	판단
1	**분할 방식 변경** — 재귀 문자 분할(함수 중간 절단) → AST 함수/클래스 단위	`0.403 → 0.389`	차이 없음
2	**메타데이터 반영** — 조각 앞에 클래스·함수 이름 헤더	`0.389 → 0.408`	미미한 경향



Qode 04 · 검색 후보 수 조정

정답 근거가 문항당 4~5개인 점을 고려해 검색 후보 수를 30으로 정했다

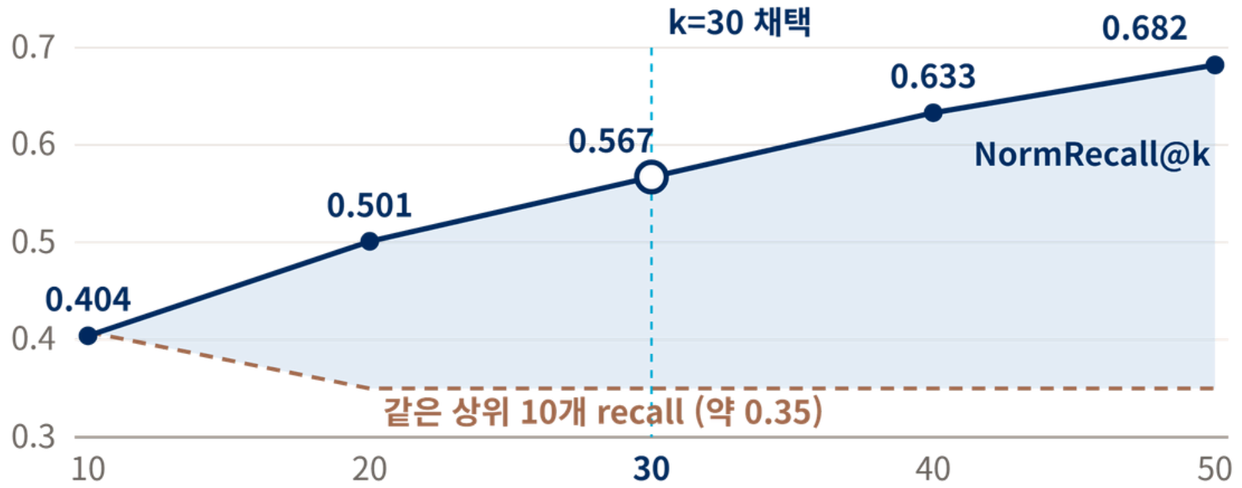
왜 늘렸나

- 문항당 정답 근거가 4~5개인데, 상위 10개 안에 모두 들어오기 어려움
- 참고한 CodeRAG 계열 프레임워크는 희소 15 + 밀집 15 + 데이터 흐름 1, 총 31개 후보 사용 [출처]

결정 · k = 30

- 후보를 늘려도 이미 찾던 정답을 놓치지 않음 (100까지 확인)
- 증가폭과 토큰 비용을 절충해 k=30으로 합의

K별 NormRecall (각자 top_k 기준)



관찰: 후보를 늘리면 상위권 순위가 흐려진다. 상위 10개 recall이 top_k=10에서 0.408, top_k=20~50에서 약 0.35. 정답이 사라지진 않지만 뒤로 밀림. 생성 단계에 주는 영향은 아직 측정하지 않음

Qode 05 · 구조 문맥 보강

코드가 놓인 구조를 문맥으로 붙이자 세밀한 위치를 더 잘 찾았다

방법

위 → 아래 · 계층 요약

모듈 → 클래스 → 함수 구조 그래프로 조각마다 자기가 속한 계층 경로를 붙임. 이름만 붙인 헤더(Qode 03)와 달리 구조 전체를 보여줌

아래 → 위 · LLM 요약

함수 → 파일 → 모듈 순서로 LLM 요약을 만들어 중간 계층에 붙임

BM25

사용 여부를 함께 비교

결과 (k=20)

계층 요약	LLM 요약	BM25	파일	심볼	줄 범위	전부 찾음	MRR
X	X	O	0.894	0.579	0.461	0.212	0.684
X	X	X	0.899	0.602	0.495	0.263	0.662
O	X	O	0.863	0.594	0.515	0.175	0.689
O	**X**	**X**	**0.869**	**0.638**	**0.564**	**0.250**	**0.697**
O	O	O	0.869	0.590	0.520	0.175	0.693
O	O	X	0.869	0.639	0.564	0.250	0.697

남색 행: 채택 구성 · 굵게: 열별 최고값

구조 문맥 보강의 효과, 남은 한계와 다음 단계

계층 요약 · 채택

줄 범위	0.495 → **0.564**
심볼	0.602 → **0.638**
파일	0.899 → **0.869** (소폭 하락)

LLM 요약 · 미채택

추가 효과 없음

비용만 늘어 제외

BM25 · 미채택

빼는 쪽이 대부분의 지표에서 높음

예외: 계층 요약 없는 조합의 MRR

남은 한계

필요한 근거를 전부 찾는 비율은 채택 구성에서 **0.250** (전체 최고 0.263)

여러 파일에 걸친 지식을 한 번에 모으는 문제가 남음

다음 단계 · 비개발자 질의 평가 (진행 중)

같은 정답을 비개발자 표현으로 물었을 때 recall이 얼마나 떨어지는지 측정하고, 업무 용어와 코드 식별자 사이의 어휘 격차를 줄이는 방법을 비교할 예정

배운 점

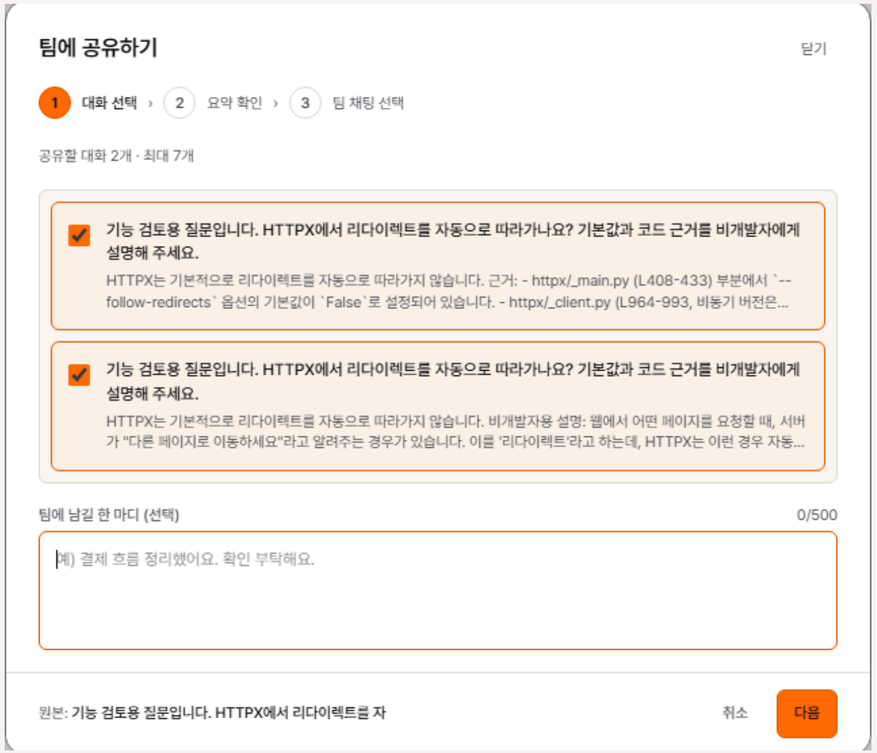
코드 지식은 조각이 아니라 구조 안에서 의미를 갖는다

정답 기준을 정의하고, 한 번에 하나씩 바꿔 원인을 좁히고, 비용과 함께 비교해야 개선 방향이 보인다

Qode 07 · 개발자 공유 기능

비개발자의 질의와 근거가 개발자에게 그대로 전달되도록 데이터 구조를 설계했다

문제 ① 비개발자가 AI로 확인한 내용을 개발자에게 다시 처음부터 설명해야 함 ② 요약만 넘기면 무엇을 근거로 한 답이었는지 사라짐



Qode · 개발자 공유 기능

데이터 구조 (개념도)

메시지	근거 스냅샷	공유 이력
질문 메시지	전체 검색 결과	원문 질문·답변 쌍
답변 메시지 + **질문 참조 칼럼**	실제 인용한 조각	요약은 공유 시점에 원본 재조회로 생성

질문과 답변을 ID로 연결

어떤 질문의 답인지 나중에도 추적

답한 시점의 근거 고정

코드가 바뀌어도 당시 무엇을 근거로 답했는지 확인

원본과 가공 결과 분리

원래 대화가 지워져도 근거가 남고, 요약 방식을 바꿔도 원본은 그대로